

# Exploring the Effectiveness of Geometrically Structured Representations for Visual Imitation

**Dylan Epstein-Gross**

Department of Computer Science  
Princeton University  
dylaneg@princeton.edu

**Arjun Menon**

Department of Computer Science  
Princeton University  
am9755@princeton.edu

**David Shustin**

Department of Computer Science  
Princeton University  
dshustin@princeton.edu

**Abstract:** Compounding distribution shift is a pervasive challenge in imitation learning on real-world robots, and robustness to it is a necessary feature for successfully cloning expert behavior on complex tasks. Recent works approach the idea of robust imitation learning by training *visual representations* on demonstration data, and using the actions of nearby encoded datapoints as a more reliable signal than the output of a learned policy. This paper takes the notion of robust representations a step further by training *geometrically structured representations* – with enforced temporal locality and isotropic Gaussian posterior – for imitation learning. The method allows us to investigate whether the action selection methods of previous works are more effective when the representations they act on are temporally localized, and also whether these action selection methods can be adjusted to stay near the prescribed posterior geometry, “biasing” a policy toward the training distribution and potentially mitigating the compounding error which is so detrimental in imitation learning. We apply our methods to an informative toy problem and a set of challenging Robomimic manipulation tasks, finding that our temporally local representations are generally more effective than prior visual representations, and that our methods of safe action selection hold promise even though they are not yet capable of reliably mitigating errors on complex environments. Geometrically structured representations of the variety considered in this paper hold potential as a starting point for further efforts in robust visual imitation.

**Keywords:** Imitation Learning, Representation Learning, Contrastive Learning, Robustness

## 1 Introduction

Offline imitation learning techniques, including behavior cloning (BC) [1], have been effective in solving complex problems in robotics. However, imitation learning suffers from compounding errors during execution as the distribution of observed states drifts from the distribution in expert trajectories [2]. One promising direction to improving imitation learning is improving *robustness* to distribution shift by learning *recovery* behaviors towards the training distribution. Several methods improve robustness by augmenting expert data to elicit recovery behavior [3, 4] or simply collecting more expert data under the policy’s own distribution [2]. Others adopt more expressive policy architectures altogether [5, 6, 7]. These approaches operate at the data or policy level, but data-level approaches require costly human effort or risk degrading demonstration quality, while policy-level approaches are largely dependent on the embedding space they operate over. Recent work has

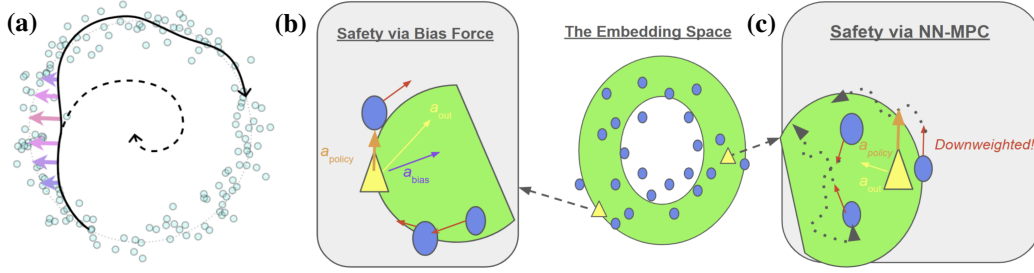


Figure 1: (a) Sketch of the broad goal of the safety mechanism in our work – we use knowledge of the training distribution geometry to push potentially diverging trajectories back into distribution. (b) Bias Force enforces actions which point toward the sphere by using an inverse dynamics model to obtain a maximally safe  $a_{bias}$ , and averaging that with the potentially unsafe  $a_{policy}$ . (c) NN-MPC performs NN action selection but downweights actions which led demo states into “risky” trajectories relative to the safety sphere, naturally leading to actions which point toward the sphere.

shown that certain representation learning methods naturally produce embeddings with rich geometric structure, specifically isotropic Gaussian distributions, whose norms concentrate around a spherical shell [8, 9, 10]. Naturally, this raises the question of whether the geometric properties of an isotropic Gaussian can be used to detect and recover from distribution shift.

We explore this direction, building on Visual Imitation through Nearest Neighbors (VINN) [11], which showed that good representations suffice for imitation learning by simple nearest-neighbor action retrieval. While VINN provides a simple recipe for extracting visual policies from representations, VINN’s BYOL representations cluster visually similar states without regard for temporal or posterior structure. In general, it is unclear what kinds of latent-space geometry are most useful for imitation learning. We consider two key questions:

1. Are **temporally local representations more effective** for imitation learning on observation embeddings?
2. Can we **exploit the geometry of representations** to produce robust imitation policies by guiding actions toward the training data distribution, as pictured in Figure 1(a)?

To answer these, we learn representations using Contrastive RL (CRL) and a Joint Embedding Predictive Architecture (JEPA), propose two geometry-aware robustness mechanisms, and evaluate on a diagnostic gridworld and Robomimic [12] manipulation tasks. Our evaluations uncover various nuances regarding both VINN and the nature of embedding geometries. We find that contrary to VINN’s results, behavior cloning consistently outperforms nearest-neighbor action extraction on some temporally local representations, and these representations produce higher-quality actions than purely visual representations on complex environments. Our safety methods successfully reduce out-of-distribution collisions in the gridworld, while not clearly improving performance on the more challenging Robomimic tasks. However, we find that deviation from the learned Gaussian geometry correlates with task failure, suggesting that the geometric signal is meaningful even when our current safety mechanisms are not yet capable enough to exploit it.

## 2 Related Work

**Robust Imitation Learning.** Robustness has been a persistent challenge in imitation learning since the earliest BC policies for self-driving cars were found to be brittle under small distribution shifts [1]. A natural solution is to expand the training data: incorporating additional camera views to synthetically broaden the training distribution [3], iteratively collecting corrective demonstrations under the current policy via DAgger [2], or automatically injecting noise via DART [4]. While effective, these data-level approaches are either costly in human effort or risk degrading demonstration quality. An orthogonal line of work addresses compounding error by augmenting the policy architecture:

Diffusion Policy [5] captures multimodal action distributions via denoising, and ACT [6] reduces the effective horizon through action chunking. Recent theory formalizes why such expressiveness helps, showing that smooth, deterministic, Markovian policies provably suffer exponential compounding error [7]. Our work takes a complementary approach, changing the *representations* rather than the data or the policy architecture to enable detection of and recovery from distribution drift.

**Representation Learning.** Self-supervised representation learning methods learn to map similar inputs to nearby latent embeddings while avoiding *collapse*, the degenerate solution where all inputs map to the same point. Early contrastive methods like SimCLR [13] prevent collapse by pushing apart embeddings of distinct images, while BYOL [14] removes the need for negative pairs by using a lagging target network. VICReg [15] takes a different route, directly regularizing embedding variance, invariance, and covariance to prevent collapse without contrastive pairs or momentum encoders.

The JEPA framework [16] generalizes these ideas by predicting future embeddings rather than reconstructing inputs, with instantiations for images [17] and video [18]. A key development for our purposes is LeJEPA [9], which introduces SIGReg, a regularizer that encourages embeddings to follow an isotropic Gaussian, grounded in the Cramér–Wold theorem. LeWorldModel [19] validates this regularization strategy for stable JEPA training from pixels in control settings. On the contrastive side, CRL [20] learns temporally structured representations via the InfoNCE loss [21], organizing embeddings by temporal reachability rather than visual similarity. CRTR [22] improves on CRL by reducing spurious features through a combinatorial negative sampling scheme. Importantly for our work, both CRL and JEPA can be regularized to produce isotropic Gaussian embeddings, a geometric property we exploit for out-of-distribution detection.

**Representations for Imitation Learning.** Representation learning has already shown promise as an aid to imitation. R3M [23] demonstrated that visual representations pre-trained on human video can enable data-efficient policy learning across manipulation tasks. VINN [11] pushed this further, showing that BYOL representations paired with simple nearest-neighbor action retrieval can outperform BC and generalize to partially-masked inputs, suggesting that good representations may be *sufficient* for imitation learning. However, VINN’s BYOL embeddings lack structure beyond grouping visually similar states: they are not organized by temporal proximity or shaped to follow a known distribution, which complicates any attempts to assign meaning to an embedding’s position relative to the distribution as a whole. Other works integrate contrastive and temporal representations with imitation [24, 25], but none directly compare temporally structured embeddings against VINN’s visual clustering using its simple action extraction recipes. We build on VINN’s core insight, asking whether representations with richer temporal and geometric structure can yield more robust policies.

**Out-of-Distribution Detection and Recovery.** A key challenge in deploying learned policies is detecting when the robot has drifted out of distribution and recovering gracefully. Reichlin et al. [26] propose a recovery policy that steers agents back to the training manifold by approximating latent data density and applying corrective actions, demonstrating that explicit recovery mechanisms can rescue policies where BC alone fails. In the broader literature, the Gaussian Annulus Theorem [10] implies that in-distribution points in a high-dimensional isotropic Gaussian concentrate near a spherical shell, a property that has been exploited for OOD detection via embedding norm deviations [27]. Our safety methods (Section 4.1) build on this principle, using the geometric structure of Gaussian-regularized representations to detect and correct for distribution shift.

### 3 Learning Structured Representations

We seek to answer the *first of our key questions* by learning representations of our expert demonstrations that are more *temporally* and *geometrically* structured than VINN’s BYOL embeddings. Of the representation methods discussed in Section 2, we focus on two: Contrastive RL (CRL) and Joint Embedding Predictive Architecture (JEPA). We choose these methods due to their prominence in the literature and the rich representations they have been shown to learn on demonstration data

in other works, as well as the fact that both provide the optionality to learn an embedding space with *isotropic Gaussian* geometry. Beyond the brief summaries given in Section 2, full architectural details and training objectives are provided in Appendix B.

## 4 Extracting Robust Actions

To answer the [first of our key questions](#), we must develop simple policies on top of the representations learned in Section 3. We use the same methods as used in VINN to enable easy comparison with the prior paper: *nearest-neighbor selection* (NN) and *behavior cloning on representations* (BC). The implementation details largely match the reference work [11] and can be found in Appendix C.

Though these two action extraction methods allow us to test whether the temporal locality of CRL/JEPA representations produces higher-fidelity actions than the visual clustering of BYOL, they do not take full advantage of the additional *geometric* structure we have induced via the isotropic Gaussian regularization. Per our [second key question](#), we explore whether we can exploit the learned geometry to guide our actions toward the training regime.

### 4.1 Safety via NN-MPC

Crucial to our analysis is the Gaussian Annulus Theorem result that the norms of points drawn from a high-dimensional isotropic Gaussian distribution concentrate around a sphere, with the probability of deviation from the sphere’s radius falling off exponentially [10]. This means that for Gaussian-regularized representations, points in the training data distribution will be encoded with norms near a sphere, while out-of-distribution points may be encoded with arbitrary norms. This provides a simple test for the likelihood that an observed state is within the training distribution: the closeness of its norm to the empirical radius of the sphere  $\hat{R}$  around which the demonstration embeddings concentrate (which we call the “safety sphere”).

As discussed in Section 2, a primary concern in imitation learning is that reaching an out-of-distribution state causes the policy to quickly behave in unexpected ways. This also holds for VINN – we would intuitively like to avoid taking actions that could lead to out-of-distribution states. We do this by “rolling out” each of the  $k$  nearest neighbors<sup>1</sup> of  $z$  into their  $H$  future states from the demonstration trajectory, and defining the score of the neighbor as the maximum squared deviation between the norms of the latent states in the neighbor’s rollout and  $\hat{R}$ :

$$\hat{R} = \frac{1}{N} \sum_{i=1}^N \|z_i\|, \quad D^{(i)} = (z^{(i)}, z_{+1}^{(i)}, z_{+2}^{(i)}, \dots, z_H^{(i)})$$

$$q^{(i)} = \max_j \left( \|D_j^{(i)}\| - \hat{R} \right)^2, \quad a_{\text{out}}(z) = \sum_{i=1}^k \left( \frac{\exp(-q^{(i)})}{\sum_{j=1}^k \exp(-q^{(j)})} \right) \cdot a^{(i)}$$

From there we run a locally-weighted regression to get output action  $a_{\text{out}}$ , using the SoftMin of the “riskiness” score  $q^{(i)}$  rather than the “distance” score used by VINN (and discussed in Appendix C). This is grounded in the fact from [10] that the probability of a point appearing in an isotropic Gaussian of sphere radius  $r$  with norm  $L$  falls off exponentially with  $(L - r)^2$ ; so our  $\exp(-q^{(i)}) / \sum \exp(-q^{(j)})$  is actually the normalized probability of the most distant state in neighbor  $i$ ’s rollout still lying within the training distribution<sup>2</sup>. Nearest Neighbor Model Predictive Control (NN-MPC) modifies NN to downweight actions which run the risk of leaving the training regime, as sketched in Figure 1(c).

<sup>1</sup>Per Appendix C, these are denoted  $(z^{(i)}, a^{(i)})$  for  $i \in [k]$ . The latent states of the larger dataset are denoted  $z_i$  for  $i \in [N]$ .

<sup>2</sup>Technically, it is guaranteed to lie within the training distribution because it is a training datapoint. However, it might lie near the boundary of this distribution, which will cause it to be downweighted (these points are less likely to have come from the overall distribution).

## 4.2 Safety via Bias Force

NN-MPC implicitly prevents failures instead of recovering from them once they have occurred, which can happen due to random noise in the environment controls. We also consider a more complex method which has the potential to recover when the observed state is already near the edge of the training distribution, by applying a “bias force” pushing back toward the safety sphere. To take an action toward an arbitrary direction in latent space, we first must learn an inverse-dynamics model, which we do using Goal-Conditioned Behavior Cloning (GCBC) [28]. Unlike the BC method described in Appendix C, which is conditioned on a single representation, this policy is conditioned on both a state representation and a goal representation:

$$\pi_{\theta}(\cdot|z, z') = \text{TanhNormal}(\mu_{\theta}(z, z'), \sigma_{\theta}(z, z')) \implies \text{at eval, } a_{\text{bias}}(z, z') = \tanh(\mu_{\theta}(z, z'))$$

Per Ding et al. [28], we train this network on batches of encoded datapoint pairs<sup>3</sup>  $(z_i, a_i, z'_i)$  via maximum-likelihood:

$$\mathcal{L}_{\text{GCBC}} = -\frac{1}{B} \sum_{i=1}^B \log(\pi_{\theta}(a_i|z_i, z'_i))$$

Thus the GCBC network learns to take actions that transition from latent states  $z$  to latent states  $z'$ , assuming those  $z$  and  $z'$  are seen close together in the demonstration dataset.

The logic of Vershynin [10] dictates that for a given latent state  $z$ , the closest “safe” point to it is its projection onto the safety sphere:  $z_{\text{safe}} = \hat{R} \cdot z / \|z\|$ . This is the target direction of our biasing action. The output action is interpolated between the base action of the policy (which goes toward the goal) and the bias action (which goes toward safety) with a strength  $\lambda_{\text{safe}}$  proportional to the normalized deviation from  $\hat{R}$ : when  $\|z\|$  is far from  $\hat{R}$ , we are far outside distribution and prioritize safety; when  $\|z\|$  is close to  $\hat{R}$ , we are within distribution and prioritize goal-reaching. Our output action is derived with<sup>4</sup>:

$$\lambda_{\text{safe}} = \min \left( k_p \cdot \left| \frac{\|z\|}{\hat{R}} - 1 \right|, 1 \right), \quad a_{\text{out}}(z) = (1 - \lambda_{\text{safe}})a_{\text{policy}}(z) + \lambda_{\text{safe}}a_{\text{bias}}(z, z_{\text{safe}})$$

where the base action  $a_{\text{policy}}$  comes from the baseline methods in Appendix C. The added complexity of an inverse-dynamics model allows us to not only prevent bad base actions from being taken, but also to integrate new actions for recovering from nearly out-of-distribution states, as sketched in Figure 1(b).

## 5 Experiments

We evaluate our method on several challenging environments requiring recovery behaviors to test the following hypotheses:

1. **Using temporally local representations results in more effective imitation learning policies**
2. **Geometry-aware action biasing toward the training distribution improves the robustness of visual imitation learning**

We evaluate three representation types: one without geometric structure (BYOL [14]), and two with temporal locality and isotropic Gaussian posterior regularization (CRL [20], JEPA [16]). Our baselines are nearest-neighbor (NN) and behavior-cloning (BC) policies trained on top of frozen representations. We test our method by biasing nearest-neighbor and behavior-cloned policies with our inverse-dynamics model, as described above (NN-Bias and BC-Bias), and we also evaluate our inverse-dynamics-agnostic biasing method NN-MPC.

<sup>3</sup>Start states and goal states are again separated by a geometric interval.

<sup>4</sup>The coefficient  $k_p$  is akin to the gain in a P-controller, governing the rate at which we correct for errors.

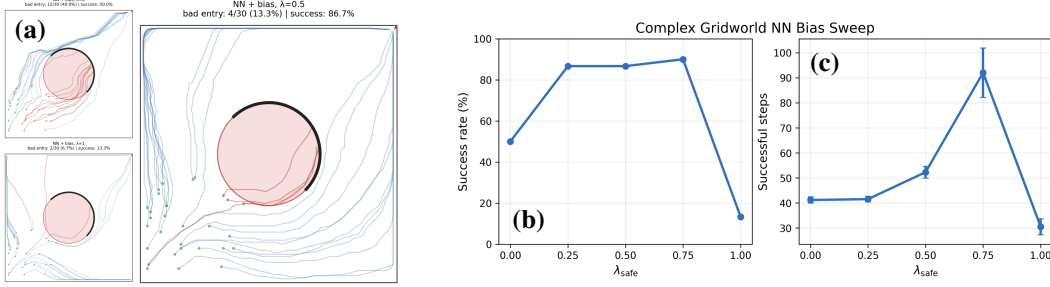


Figure 2: (a) Gridworld and rollouts under NN-Bias for different fixed biasing strengths. Clockwise from top-left:  $\lambda_{\text{safe}} = 0$ ,  $\lambda_{\text{safe}} = 0.5$ , and  $\lambda_{\text{safe}} = 1$ . Increasing  $\lambda_{\text{safe}}$  causes the policy to avoid the excluded region, but eventually degrades performance as the influence of the goal-reaching policy vanishes. (b) Success rate as a function of biasing strength  $\lambda_{\text{safe}}$ . Success increases with  $\lambda_{\text{safe}}$  until  $\lambda_{\text{safe}} = 1$ , where the guidance of the imitation policy vanishes entirely. (c) Mean trajectory length for successful rollouts. As  $\lambda_{\text{safe}}$  increases, successful rollouts grow longer because the actor takes paths farther from the excluded region.

## 5.1 Toy Problem

**Gridworld construction.** We evaluate our methods on a gridworld environment with high amounts of noise and a missing part of the training distribution – the exact type of task which would typically cause compounding error in naive imitation learning. We designed a 2D gridworld with pixel-based observations in which the actor moves a black pixel on a white background with 2D continuous controls mapping to translation with additive Gaussian noise. The expert moves from the initial point to the top-right corner by following a sequence of randomly sampled waypoints, while avoiding a circular excluded zone, as shown in Figure 2(a). The excluded zone is mutually exclusive with the support of the training distribution. Normal imitation learning may lead the agent along a demonstration that nears the circle and accidentally enters it, at which point correct actions are difficult to predict because states within the circle are outside the training distribution, causing compounding error that traps the agent behind the semi-circular wall. We expect our biasing methods to encourage imitation policies to follow expert demonstrations while also navigating away from the observed “hole” in the training distribution, avoiding the excluded zone and maximizing success rate.

**Increased biasing increases success rate.** We measure the success rate of biasing with several strengths using CRL embeddings. We observe that increasing the biasing strength  $\lambda_{\text{safe}}$  increases success rate on gridworld, as shown in Figure 2(b). However, as  $\lambda_{\text{safe}}$  approaches 1, the success rate collapses to zero, as the influence of the expert demonstrations disappears and the agent only seeks to avoid the excluded region without seeking to reach the goal. As the biasing strength increases, the mean number of steps per successful trajectory increases as the actor takes more circuitous routes avoiding the excluded region – exemplifying the tradeoff between safety and efficiency.

**Biasing decreases excluded zone collisions.** We assess the effectiveness of biasing for staying in the training data distribution by measuring the rate at which the actor enters the excluded zone. We find that with increasing  $\lambda_{\text{safe}}$ , not only does success rate increase, but collisions with the excluded zone decrease, as shown in Figure 2(a). We conclude that the inverse dynamics model is successfully *learning to avoid the excluded zone* and stay in the expert demonstration distribution. Full results for this statistic across all methods are included in Appendix D, and reflect the same pattern of biasing being generally effective for increasing success rate and time spent in the training distribution.

## 5.2 Robomimic Environments

We evaluate our methods on a realistic simulation of robotic manipulation tasks. We chose the Robomimic [12] environment because it is a popular benchmark including a large offline imitation learning dataset. Robomimic also includes a data augmentation pipeline that replays expert trajectories under geometric transforms, which would be useful for future experiments like scaling the

| Method  | Lift (% Success $\uparrow$ ) |      |      | Can (% Success $\uparrow$ ) |             |      | Square (% Success $\uparrow$ ) |             |      |
|---------|------------------------------|------|------|-----------------------------|-------------|------|--------------------------------|-------------|------|
|         | BYOL                         | CRL  | JEPA | BYOL                        | CRL         | JEPA | BYOL                           | CRL         | JEPA |
| NN      | 77.0                         | 2.0  | 31.0 | 0.5                         | 0.0         | 0.0  | 1.5                            | 0.0         | 6.0  |
| BC      | <b>90.5</b>                  | 76.5 | 20.5 | 0.0                         | <b>51.0</b> | 0.0  | 2.5                            | <u>19.0</u> | 3.0  |
| NN-MPC  | 30.0                         | 3.0  | 12.0 | 0.5                         | 0.0         | 0.0  | 3.0                            | 0.0         | 1.5  |
| NN-Bias | 85.5                         | 4.5  | 22.5 | 1.0                         | 7.0         | 0.0  | 2.5                            | 0.0         | 3.5  |
| BC-Bias | <u>90.0</u>                  | 68.5 | 22.0 | 0.0                         | <u>43.0</u> | 0.0  | 4.0                            | <b>24.0</b> | 3.5  |

Table 1: Success rates (over 200 rollouts) for all embedding types and environments on baselines and biasing methods. The best figure is in **bold**, second-best figure is underlined.

number of demonstrations, learning more effective inverse dynamics models, and excluding demonstrations that pass through a region of space.

We test each method on the *Lift*, *Can*, and *Square* tasks from Robomimic. For summaries of these environments and rendered frames see Appendix E.

**Behavior cloning beats NN methods.** Contrary to VINN [11], we find that behavior cloning on frozen representations outperforms locally-weighted regression. BC methods generally outperform NN methods in Table 1, even when biasing is added. This could be because Robomimic tasks involve dextrous manipulation, which is more difficult than the handle and door-opening tasks on which VINN evaluates success rates. We examine why BC performs so well while NN has a near-zero success rate for CRL specifically in Appendix F, finding that the evidence points toward a subtle difference between the latent geometries of CRL and BYOL.

**CRL outperforms BYOL and JEPA embeddings.** The results in Table 1 and Figure 3(a) show that CRL generally performs better than JEPA or BYOL on Robomimic tasks. For Can and Lift, CRL is the only method that achieves double-digit success rate percentages. This suggests that *temporal locality is helpful for imitation learning* on frozen representations. BYOL performs the best on Lift, which might be because Lift is shorter horizon than Can and requires less skill composition than Square.

**Biasing is not clearly helpful.** Our results show that, in aggregate, *biasing does not result in more robust policies*. While biasing helps avoid the “excluded zone” in our toy problem, results in Table 1 and Figure 3(b) show that biasing NN or BC is not consistently better than the baseline. Biased methods are not significantly more successful than their non-biased counterparts across most embedding types and tasks, though there is a visible improvement between BC and BC-Bias in the more challenging Square task. Also, NN-MPC *decreases* the performance of JEPA-NN on Lift<sup>5</sup>. In Appendix F, we explore why this occurs. Still, there are some trajectories in which biasing is qualitatively helpful, which we visualize in Appendix E.

**The geometry of success.** We find that deviation from the safety sphere is correlated with failure. By measuring the deviation from the expected geometry, we find that failure trajectories tend to deviate from the sphere more often than successes<sup>6</sup>, as shown in Table 2. This result holds only for methods with Gaussian posterior (such as CRL), suggesting that geometric structure can be a tool for producing robust policies.

This result implies directions for improving our method by *strengthening recovery behavior* elicited by the inverse-dynamics model [26].

<sup>5</sup>BYOL-NN-MPC is also significantly worse than BYOL-NN on Lift, but this is expected because NN-MPC’s scoring function is not biasing towards anything meaningful on BYOL’s non-Gaussian posterior.

<sup>6</sup>This pattern holds almost entirely in the table, except in NN-Bias for Can. However, this is likely due to the fact that NN-Bias succeeds very few times, so the numbers are skewed by low sample size.

| Outcome | Lift (% Deviation) |         | Can (% Deviation) |         | Square (% Deviation) |         |
|---------|--------------------|---------|-------------------|---------|----------------------|---------|
|         | Success            | Failure | Success           | Failure | Success              | Failure |
| NN      | 0.0%               | 30.1%   | N/A               | 74.5%   | N/A                  | 64.5%   |
| BC      | 39.9%              | 74.5%   | 46.1%             | 63.3%   | 2.6%                 | 49.4%   |
| NN-MPC  | 0.0%               | 19.6%   | N/A               | 69.0%   | N/A                  | 71.0%   |
| NN-Bias | 11.1%              | 30.9%   | 85.7%             | 68.3%   | N/A                  | 53.0%   |
| BC-Bias | 30.7%              | 79.4%   | 44.2%             | 72.8%   | 18.8%                | 44.7%   |

Table 2: Deviation rates (over 200 rollouts) for various methods based on the CRL representations, categorized by whether the rollout was a success or a failure. A rollout “deviates” if it contains a latent state with  $> 10\%$  difference from sphere radius  $\hat{R}$ . Cells with no successes are labelled “N/A.”

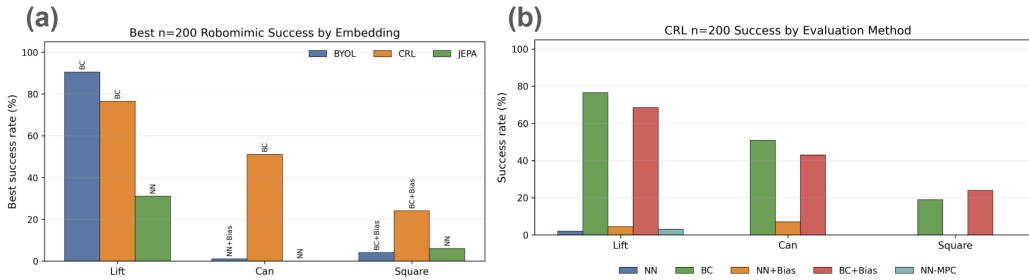


Figure 3: (a) Comparison of the best success rate for each representation method on each of the three Robomimic tasks, resolving the **first key question**. (b) Comparison of the success rate of the five different action selection methods for CRL embeddings on each of the three Robomimic tasks, resolving the **second key question**.

## 6 Conclusions

In this work, we test several hypotheses about the geometry of latent representations for imitation learning. We develop methods that exploit the expected geometry of “in-distribution” data to elicit recovery behavior during imitation learning. We test these methods on challenging robotic simulation tasks and answer the key questions we posed at the start. We show that **temporally local representations can be more effective for imitation learning**, with CRL significantly outperforming BYOL on the most challenging tasks – but it seems that *only behavior cloning* is able to extract this effectiveness. Nearest-neighbor locally-weighted regression, as proposed by Pari et al. [11], struggles to handle the shape of CRL embeddings, which are clustered by trajectory and have great distances between them. However, **our attempts of biasing towards the geometry of expert demonstration representations are largely ineffective** on complex, high-dimensional tasks, though their intended effect is visible in simple toy environments and on individual rollouts. Nonetheless, we provide evidence that structured geometry is a useful signal for eliciting recovery behavior and point toward new directions to strengthen our result.

## 7 Limitations

**Representation Learning.** While our CRL representations do successfully produce the temporal structure and norm clustering we had hoped for (see Appendix E), they introduce their own concerns. The dual gradient optimization potentially makes training unstable, and it is difficult to balance the hyperparameters (such as the repetition factor) to obtain good temporal clustering. The failure of NN on CRL representations may be due to within-trajectory clustering, which causes all neighbors to be

drawn from the same demonstration rather than from across different demonstrations (as discussed in Appendix F). By comparison, BYOL requires little tuning. The failure of NN on CRL could also be due to contrastive learning not producing a consistent distance quasimetric, which might make nearest-neighbors ineffective; this could be solved by integrating the more recent Contrastive Metric Distillation, which contrastively learns a guaranteed quasimetric [29]. Future research may investigate more hyperparameter combinations or alternative representation schemes that are more robust to parameter selection and avoid in-trajectory clustering.

**Action Biasing.** While our embeddings do produce the desired safety sphere, our action biasing methods are not always good at staying on that sphere. Our added bias force is more heuristic than rigorous, and tuning the  $k_p$  gain is difficult. Since the inverse-dynamics policy  $a_{\text{bias}}$  is only trained on expert demonstrations, it may struggle to perform recovery behavior even when  $\lambda_{\text{safe}}$  is large. We also see that many failures still occur even without significant norm deviations, suggesting that many OOD states lie near the sphere, so biasing toward it is not always safe. Future work may consider more rigorous control methods for staying near the sphere, use recovery-inducing methods like DART [4] to train a more robust inverse-dynamics model, or incorporate limited failure data to improve the safety sphere’s discrimination between in-distribution and out-of-distribution states.

**Evaluations.** The environments on which we evaluate are also limited. Though they are challenging, Robomimic environments lack realism, and methods that work well on them may not generalize to the real world. They also do not allow easy masking/filtering in the way that we filter Gridworld or in the way that the VINN authors semantically masked their real-world push environment [11], limiting our ability to study robustness and real-world generalization. Future work may consider applying our structured representations and biased actions to real-world tasks, where the robustness advantages may be more apparent than in the relatively predictable Robomimic simulation.

## 8 Contributions

This project was made possible through the equal contributions of all three co-first authors, who are listed in alphabetical order.

**Dylan Epstein-Gross** developed Gridworld toy problem, implemented and tuned CRL embeddings, came up with the bias force mechanism, implemented biasing strategies, ran experiments/ablations and generated visualizations, wrote key methodology sections of paper and worked on the rest.

**Arjun Menon** formulated, implemented, and tuned task-specific JEPA embeddings; formulated and implemented broader selection score calculation to evaluate embedding coverage; wrote key background and methodology sections of paper and worked on the rest.

**David Shustín** replicated the VINN paper, implemented and tuned BYOL embeddings, came up with NN-MPC biasing, wrote background and experiment sections of paper and worked on the rest.

### Acknowledgments

We would like to thank Professor Dhruv Shah, Asher Hancock, and Lihan Zha for their invaluable guidance on this project. We also thank Professor Ben Eysenbach for facilitating various RL discussions last semester that influenced this work.

### References

- [1] D. A. Pomerleau. ALVINN: An Autonomous Land Vehicle in a Neural Network. In *Advances in Neural Information Processing Systems*, volume 1. Morgan-Kaufmann, 1988. URL <https://proceedings.neurips.cc/paper/1988/hash/812b4ba287f5ee0bc9d43bbf5bbe87fb-Abstract.html>.
- [2] S. Ross, G. J. Gordon, and J. A. Bagnell. A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning.
- [3] M. Bojarski, D. D. Testa, D. Dworakowski, B. Firner, B. Flepp, P. Goyal, L. D. Jackel, M. Monfort, U. Muller, J. Zhang, X. Zhang, J. Zhao, and K. Zieba. End to End Learning for Self-Driving Cars, Apr. 2016. URL <http://arxiv.org/abs/1604.07316>. arXiv:1604.07316 [cs].
- [4] M. Laskey, J. Lee, R. Fox, A. Dragan, and K. Goldberg. DART: Noise Injection for Robust Imitation Learning, Oct. 2017. URL <http://arxiv.org/abs/1703.09327>. arXiv:1703.09327 [cs].
- [5] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. Burchfiel, and S. Song. Diffusion Policy: Visuomotor Policy Learning via Action Diffusion. In *Robotics: Science and Systems (RSS)*, 2023.
- [6] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn. Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware. In *Robotics: Science and Systems (RSS)*, 2023.
- [7] M. Simchowitz, D. Pfrommer, and A. Jadababae. The Pitfalls of Imitation Learning when Actions are Continuous, July 2025. URL <http://arxiv.org/abs/2503.09722>. arXiv:2503.09722 [cs].
- [8] B. Eysenbach, V. Myers, R. Salakhutdinov, and S. Levine. Inference via Interpolation: Contrastive Representations Provably Enable Planning and Inference. *Advances in Neural Information Processing Systems*, 37:58901–58928, Dec. 2024. doi:10.52202/079017-1878. URL [https://proceedings.neurips.cc/paper\\_files/paper/2024/hash/6c49d2ad55e50c5ebc1002fdc50e48e5-Abstract-Conference.html](https://proceedings.neurips.cc/paper_files/paper/2024/hash/6c49d2ad55e50c5ebc1002fdc50e48e5-Abstract-Conference.html).

- [9] R. Balestriero and Y. LeCun. LeJEPa: Provable and Scalable Self-Supervised Learning Without the Heuristics, Nov. 2025. URL <http://arxiv.org/abs/2511.08544>. arXiv:2511.08544 [cs].
- [10] R. Vershynin. *High-Dimensional Probability: An Introduction with Applications in Data Science*. Cambridge University Press, Sept. 2018. ISBN 978-1-108-24454-1. Google-Books-ID: TahxDwAAQBAJ.
- [11] J. Pari, N. M. Shafiullah, S. P. Arunachalam, and L. Pinto. The Surprising Effectiveness of Representation Learning for Visual Imitation, Dec. 2021. URL <http://arxiv.org/abs/2112.01511>. arXiv:2112.01511 [cs].
- [12] A. Mandlekar, D. Xu, J. Wong, S. Nasiriany, C. Wang, R. Kulkarni, L. Fei-Fei, S. Savarese, Y. Zhu, and R. Martín-Martín. What Matters in Learning from Offline Human Demonstrations for Robot Manipulation, Sept. 2021. URL <http://arxiv.org/abs/2108.03298>. arXiv:2108.03298 [cs].
- [13] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton. A Simple Framework for Contrastive Learning of Visual Representations. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, pages 1597–1607, 2020.
- [14] J.-B. Grill, F. Strub, F. Altché, C. Tallec, P. H. Richemond, E. Buchatskaya, C. Doersch, B. A. Pires, Z. D. Guo, M. G. Azar, B. Piot, K. Kavukcuoglu, R. Munos, and M. Valko. Bootstrap your own latent: A new approach to self-supervised Learning, Sept. 2020. URL <http://arxiv.org/abs/2006.07733>. arXiv:2006.07733 [cs].
- [15] A. Bardes, J. Ponce, and Y. LeCun. VICReg: Variance-Invariance-Covariance Regularization for Self-Supervised Learning. In *International Conference on Learning Representations (ICLR)*, 2022. doi:10.48550/arXiv.2105.04906. URL <https://arxiv.org/abs/2105.04906>.
- [16] Y. LeCun. A Path Towards Autonomous Machine Intelligence, June 2022. URL <https://openreview.net/forum?id=BZ5a1r-kVsf>. Version 0.9.2.
- [17] M. Assran, Q. Duval, I. Misra, P. Bojanowski, P. Vincent, M. Rabbat, Y. LeCun, and N. Ballas. Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 15619–15629, 2023.
- [18] A. Bardes, Q. Garrido, J. Ponce, X. Chen, M. Rabbat, Y. LeCun, M. Assran, and N. Ballas. Revisiting Feature Prediction for Learning Visual Representations from Video, 2024. URL <http://arxiv.org/abs/2404.08471>. arXiv:2404.08471.
- [19] L. Maes, Q. L. Lidec, D. Scieur, Y. LeCun, and R. Balestriero. LeWorldModel: Stable End-to-End Joint-Embedding Predictive Architecture from Pixels, Mar. 2026. URL <http://arxiv.org/abs/2603.19312>. arXiv:2603.19312 [cs].
- [20] B. Eysenbach, T. Zhang, S. Levine, and R. R. Salakhutdinov. Contrastive Learning as Goal-Conditioned Reinforcement Learning. *Advances in Neural Information Processing Systems*, 35:35603–35620, Dec. 2022. URL [https://proceedings.neurips.cc/paper\\_files/paper/2022/hash/e7663e974c4ee7a2b475a4775201ce1f-Abstract-Conference.html](https://proceedings.neurips.cc/paper_files/paper/2022/hash/e7663e974c4ee7a2b475a4775201ce1f-Abstract-Conference.html).
- [21] A. v. d. Oord, Y. Li, and O. Vinyals. Representation Learning with Contrastive Predictive Coding, 2018. URL <http://arxiv.org/abs/1807.03748>. arXiv:1807.03748.
- [22] A. Ziarko, M. Bortkiewicz, M. Zawalski, B. Eysenbach, and P. Milos. Contrastive Representations for Temporal Reasoning, Sept. 2025. URL <http://arxiv.org/abs/2508.13113>. arXiv:2508.13113 [cs].

- [23] S. Nair, A. Rajeswaran, V. Kumar, C. Finn, and A. Gupta. R3M: A Universal Visual Representation for Robot Manipulation. In *Conference on Robot Learning (CoRL)*, pages 892–909, 2022.
- [24] Y. Wang, L. Tao, B. Du, Y. Lin, and C. Xu. Visual Imitation Learning with Calibrated Contrastive Representation, Jan. 2024. URL <http://arxiv.org/abs/2401.11396>. arXiv:2401.11396 [cs].
- [25] O. Nachum and M. Yang. Provable Representation Learning for Imitation with Contrastive Fourier Features, Oct. 2021. URL <http://arxiv.org/abs/2105.12272>. arXiv:2105.12272 [cs].
- [26] A. Reichlin, G. L. Marchetti, H. Yin, A. Ghadirzadeh, and D. Kragic. Back to the Manifold: Recovering from Out-of-Distribution States. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 1992–1999, 2023.
- [27] G. Osada, T. Takahashi, B. Ahrary, and T. Nishi. Out-of-Distribution Detection with Reconstruction Error and Typicality-Based Penalty. In *IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pages 5765–5774, 2023.
- [28] Y. Ding, C. Florensa, M. Phielipp, and P. Abbeel. Goal-conditioned Imitation Learning, May 2020. URL <http://arxiv.org/abs/1906.05838>. arXiv:1906.05838 [cs].
- [29] V. Myers, C. Zheng, A. Dragan, S. Levine, and B. Eysenbach. Learning Temporal Distances: Contrastive Successor Features Can Provide a Metric Structure for Decision-Making, Mar. 2025. URL <http://arxiv.org/abs/2406.17098>. arXiv:2406.17098 [cs].
- [30] A. Dawid and Y. LeCun. Introduction to Latent Variable Energy-Based Models: A Path Towards Autonomous Machine Intelligence. *Journal of Statistical Mechanics: Theory and Experiment*, 2024(10):104011, Oct. 2024. ISSN 1742-5468. doi:10.1088/1742-5468/ad292b. URL <http://arxiv.org/abs/2306.02572>. arXiv:2306.02572 [cs].

## A Code

Our full repository, including code for all our methods, specific parameter values, and experiment configs, is made public at the following Github link: <https://github.com/Dseg123/ROB534.git>. This can be used both to replicate our experiments, and as a fully functional starting point for future research into robust imitation learning via geometrically structured representations.

Due to size limits, training artifacts (weights of the embedding networks) can be found at the following Google Drive Link: <https://drive.google.com/drive/folders/1kMB8Cf7BxjNCrpuB1Jhf3sa41FjTrCHB?usp=sharing>. Visualizations of experiments (including plots and gifs), and spreadsheets of results used to generate the tables in this paper, can be found at the following Google Drive Link: [https://drive.google.com/drive/folders/1igz23BB0Phv\\_5W6CDtBm\\_0C5aBcNx1cb?usp=drive\\_link](https://drive.google.com/drive/folders/1igz23BB0Phv_5W6CDtBm_0C5aBcNx1cb?usp=drive_link). All data/environment specifications can be downloaded easily from the Robomimic website: <https://robomimic.github.io/docs/datasets/overview.html> [12].

AI-assisted tools (Claude and Codex) were used during this work for coding assistance, debugging, and generating diagrams. All experimental design, analysis, interpretation, and writing are our own.

## B Learning Structured Representations – Implementation Details

The following sections explain how our CRL and JEPA representations are implemented, elaborating on Section 3. For specific parameters, see code in Appendix A.

## B.1 CRL Representations

We base our CRL representations on the offline learning scheme discussed in [20], with a few modifications. Rather than learning a state-action encoder, we only learn a *state encoder*  $\phi(s)$  alongside the *goal encoder*  $\psi(g)$ . Drawing from the encoder architecture of VINN, each of these consists of a ResNet-50 backbone – pretrained on ImageNet – with an MLP head, taking an  $84 \times 84$  RGB image as input and emitting a 64-dimensional embedding as output [11].

We sample batches from the demonstration data using the *Contrastive Representations for Temporal Reasoning (CRTR) scheme* of Ziarko et al. [22]: each batch consists of  $B/R$  uniformly sampled trajectories, with  $R$  state-goal pairs  $(s_i, s_i^+)$  drawn at a geometric offset<sup>7</sup>.

Then, we train our encoders using a *symmetrized InfoNCE objective* of Eysenbach et al. [8]:

$$\mathcal{L}_{\text{InfoNCE}} = \sum_{i=1}^B \log \frac{e^{-\frac{1}{2}\|\phi(s_i) - \psi(s_i^+)\|_2^2}}{\sum_{j \neq i} e^{-\frac{1}{2}\|\phi(s_i) - \psi(s_j^+)\|_2^2}} + \log \frac{e^{-\frac{1}{2}\|\phi(s_i) - \psi(s_i^+)\|_2^2}}{\sum_{j \neq i} e^{-\frac{1}{2}\|\phi(s_j) - \psi(s_i^+)\|_2^2}} \quad (1)$$

This loss function encourages  $\phi(s_i)$  to lie geometrically close to  $\psi(s_i^+)$  in the latent space, and far from other  $\psi(s_j^+)$  – states end up near those that are most “temporally close” to them, rather than those that are most “visually similar” to them (the purpose of BYOL).

But as discussed in Section 1, temporal structure is not our only goal – we also want geometric structure which we can later use to bias our actions. Again drawing from Eysenbach et al. [8], we add an *L2 regularization tuned via dual gradient descent*<sup>8</sup>:

$$\mathcal{L}_{\text{Reg}} = \underbrace{\text{sg}(\lambda) \left( \frac{1}{2B} \sum_{i=1}^B (\|\phi(s_i)\|_2^2 + \|\psi(s_i^+)\|_2^2) \right)}_{\text{L2 Regularization}} + \underbrace{\lambda \left( c - \text{sg} \left[ \frac{1}{2B} \sum_{i=1}^B (\|\phi(s_i)\|_2^2 + \|\psi(s_i^+)\|_2^2) \right] \right)}_{\text{Lagrangian Dual Loss}} \quad (2)$$

This has the effect of encouraging the expected L2 norm of our embeddings to be close to our chosen value of  $c$ . Eysenbach et al. [8] argue that this should theoretically produce representations which follow an isotropic Gaussian distribution  $p(\psi) = \mathcal{N}(\psi; \mu = 0, \sigma = c \cdot I)$ . This is an important claim that we will verify experimentally and utilize in Section 4. Overall, our CRL embeddings are trained via gradient descent to optimize

$$\mathcal{L}_{\text{CRL}} = \mathcal{L}_{\text{InfoNCE}} + \mathcal{L}_{\text{Reg}}$$

over CRTR-sampled batches.

## B.2 JEPA Representations

Our JEPA representations are based on the Joint Embedding Predictive Architecture of LeCun [16], adapted from visual self-supervised learning to robotic demonstration data. Like CRL, the encoder uses an ImageNet-pretrained ResNet-50 backbone with a learned projection and fusion head [11], producing a 256-dimensional embedding  $z = f_\theta(o)$  from each  $84 \times 84$  RGB observation. Unlike CRL, JEPA does not use separate state and goal encoders. A single encoder maps all observations to a shared latent space, while a separate *predictor* network  $g_\phi$ , a 3-layer MLP conditioned on a learned horizon embedding  $h_{\text{emb}} \in \mathbb{R}^{64}$ , predicts future latent representations from the current one. The predictor is discarded after training, with only the encoder being retained for downstream use. This asymmetry, reminiscent of BYOL’s online-target split [14] but without its exponential moving average, allows the predictor module to absorb any learnable temporal dynamics, freeing the encoder to learn general-purpose state representations rather than ones specialized for prediction [16, 30].

<sup>7</sup>Specifically,  $s_i$  is drawn from step  $t$ , sampled uniformly at random, and  $s_i^+$  is drawn from step  $t' \sim t + \text{Geom}(1 - \gamma)$

<sup>8</sup>This is done via stop-gradients, shown in the equation as “sg”. Both  $\lambda$  and the encoders are trainable parameters, the former trained by Lagrangian Dual Loss and the latter trained by L2 Regularization loss.

Each training sample consists of a current observation  $o_t$ , sampled uniformly over all valid timesteps across all demonstrations, alongside future observations  $\{o_{t+h}\}$  at fixed horizons  $h \in \{1, 2, 4, 8\}$ . Unlike CRL’s geometric offset sampling, JEPA uses deterministic multi-horizon targets: each sample always produces predictions at all four horizons simultaneously, giving the encoder a multi-scale view of temporal evolution.

We use a multi-horizon latent prediction loss as our primary training signal. For each horizon  $h$ , the predictor outputs  $\hat{z}_{t+h} = g_\phi(z_t, h_{\text{emb}})$  and the encoder produces the target  $z_{t+h} = f_\theta(o_{t+h})$  with stop-gradient. The squared L2 distance between L2-normalized vectors:

$$\ell_h = \left\| \frac{\hat{z}_{t+h}}{\|\hat{z}_{t+h}\|_2} - \frac{\text{sg}(z_{t+h})}{\|\text{sg}(z_{t+h})\|_2} \right\|_2^2 \quad (3)$$

which is equivalent to  $2(1 - \cos(\hat{z}_{t+h}, z_{t+h}))$ , provides a scale-invariant per-horizon loss focused on directional agreement in latent space. The total prediction loss is calculated as a weighted average across horizons:

$$\mathcal{L}_{\text{pred}} = \frac{\sum_h w_h \cdot \ell_h}{\sum_h w_h}, \quad w = \{1:1.0, 2:1.0, 4:0.75, 8:0.5\} \quad (4)$$

with decreasing weights at longer horizons reflecting their greater prediction uncertainty. This loss encourages the encoder to capture the information necessary to predict the future latent state.

Prediction-only training is susceptible to *representation collapse*, where the encoder maps all inputs to a single point or low-dimensional subspace, trivially achieving zero prediction error. We prevent this with two complementary regularizers that, together, encourage the embedding distribution to approximate an isotropic Gaussian, the same geometric target as CRL’s L2 regularization, but achieved without needing negative labels for a contrastive approach. This two-term regularization strategy follows the approach validated by Maes et al. [19] for stable end-to-end JEPA training from pixels in control settings.

The first is a *Sketched Isotropic Gaussian Regularization* (SIGReg) loss [9]. The batch of anchor embeddings  $z_t$  is centered and projected onto 256 random unit-norm directions, and the 1D marginals are penalized for deviating from a standard normal by matching their first, second, and fourth moments:

$$\mathcal{L}_{\text{SIGReg}} = \underbrace{\mathbb{E}_k[\mu_k^2]}_{\text{mean} \rightarrow 1\text{st}} + \underbrace{\mathbb{E}_k[(\sigma_k - 1)^2]}_{\text{std} \rightarrow 2\text{nd}} + 0.1 \cdot \underbrace{\mathbb{E}_k[(\kappa_k - 3)^2]}_{\text{kurtosis} \rightarrow 4\text{th}} \quad (5)$$

where  $\mu_k$ ,  $\sigma_k$ , and  $\kappa_k$  are the mean, standard deviation, and kurtosis of the projected batch along random direction  $k$ . Balestriero and LeCun [9] determines that if *all* 1D marginals are Gaussian, then by the Cramér–Wold theorem [10] the joint distribution must be Gaussian, allowing SIGReg to enforce a global distributional constraint through cheap 1D moment matching.

The second is a per-dimension variance hinge penalty in the style of VICReg [15]:

$$\mathcal{L}_{\text{var}} = \frac{1}{D} \sum_{d=1}^D \text{ReLU}(1 - \sigma_d) \quad (6)$$

where  $\sigma_d$  is the standard deviation of dimension  $d$  across the batch. This directly prevents any individual latent dimension from collapsing, complementing SIGReg’s distributional regularization along random projections by allowing SIGReg to control the overall distributional shape, while  $\mathcal{L}_{\text{var}}$  ensures that no individual axis is underutilized.

The JEPA embeddings are trained via gradient descent to optimize

$$\mathcal{L}_{\text{JEPA}} = \mathcal{L}_{\text{pred}} + \lambda_{\text{sig}} \cdot \mathcal{L}_{\text{SIGReg}} + \mathcal{L}_{\text{var}} \quad (7)$$

with  $\lambda_{\text{sig}} = 1.0$ , using AdamW with cosine-annealed learning rate and differential learning rates for the pretrained backbone ( $0.03 \times$  base rate).

## C Extracting Robust Actions – Implementation Details

The following sections explain how our NN and BC action extractions are implemented, elaborating on Section 4. For specific parameters, see code in Appendix A.

### C.1 Nearest-Neighbor Selection

This is the core method of VINN, and we replicate it exactly. We first encode the states of all the demonstration datapoints  $((s_i, a_i) \text{ for } i = 1 \dots N)$  using our learned embedding network, forming a dataset  $Z = \{(z_i, a_i) \mid \forall i \in [N]\}$ . Then, for a given state  $s$ , we encode<sup>9</sup> it to a latent state  $z$ . We find the  $k$  nearest neighbors by Euclidean latent distance  $\|z - z_i\|_2$ , which we denote  $(z^{(1)}, a^{(1)}), (z^{(2)}, a^{(2)}) \dots (z^{(k)}, a^{(k)})$ . We then assign each of these neighbors a score by how close they are to  $z$ , and use the SoftMin of these scores to obtain the output action via locally-weighted regression:

$$q^{(i)} = \|z - z^{(i)}\|_2, \quad a_{\text{policy}}(z) = \sum_{i=1}^k \left( \frac{\exp(-q^{(i)})}{\sum_{j=1}^k \exp(-q^{(j)})} \right) \cdot a^{(i)}$$

While seemingly simple, Pari et al. [11] show that with sufficiently informative representations, actions obtained with this recipe can solve challenging tasks.

### C.2 BC on Representations

This is a baseline considered in VINN. It is more complicated than NN – requiring training an additional behavior-cloning model rather than just averaging existing actions – and Pari et al. [11] find it to perform worse than NN on most environments. However, it is unclear if the action extraction recipes that work well on BYOL representations generalize to the differently structured CRL/JEPA representations, so we consider BC as a potential alternative. Following Pari et al. [11], we condition our behavior-cloning model on the learned representations rather than raw states (which they find to be much less effective). The representations should do most of the hard work, with the BC model being a much smaller MLP that is trained for less time. The policy network outputs the  $\mu$  and  $\sigma$  from a Gaussian, and at eval time it samples from the Gaussian deterministically by drawing the mean:

$$\pi_{\theta}(\cdot|z) = \text{TanhNormal}(\mu_{\theta}(z), \sigma_{\theta}(z)) \implies \text{at eval, } a_{\text{policy}}(z) = \tanh(\mu_{\theta}(z))$$

Per the BC literature [28], we train this network on batches of encoded datapoints  $(z_i, a_i)$  via maximum-likelihood:

$$\mathcal{L}_{\text{BC}} = -\frac{1}{B} \sum_{i=1}^B \log(\pi_{\theta}(a_i|z_i))$$

Thus the BC network learns to imitate the expert demonstrations, conditioned on their representations rather than their raw states.

## D Results on Gridworld

The full results on the Gridworld toy problem are in Table 3.

The CRL results show the same trend discussed in Section 5: BC performs better than NN, and bias force on either method performs better than the baselines. When biased toward the training regime, the agent significantly increases its success rate by avoiding the excluded region, at the expense of taking a higher number of steps. NN-MPC performs poorly despite its safety mechanism, which is discussed further in Appendix F.

<sup>9</sup>Relative to BYOL and JEPA, which have a single encoder, CRL is ambiguous here because there are two encoders. We choose to encode  $z$  with the state encoder and  $Z$  with the goal encoder, since the CRL latent space only places guarantees on the distances between  $\phi(s)$  and  $\psi(s')$ , not between  $\phi(s)$  and  $\phi(s')$ .

| Embedding  | Method         | Success rate ( $\uparrow$ ) | Successful steps ( $\downarrow$ ) |
|------------|----------------|-----------------------------|-----------------------------------|
| BYOL       | NN             | 75.0%                       | $41.9 \pm 0.4$                    |
| BYOL       | BC             | 32.5%                       | $29.5 \pm 0.3$                    |
| BYOL       | NN-Bias        | 45.5%                       | $33.3 \pm 0.3$                    |
| BYOL       | BC-Bias        | 33.0%                       | $29.6 \pm 0.3$                    |
| BYOL       | NN-MPC         | 69.5%                       | $44.1 \pm 0.6$                    |
| CRL        | NN             | 58.5%                       | $41.6 \pm 0.4$                    |
| CRL        | BC             | 79.0%                       | $31.4 \pm 0.2$                    |
| CRL        | NN-Bias        | 78.5%                       | $45.0 \pm 0.6$                    |
| <b>CRL</b> | <b>BC-Bias</b> | <b>87.5%</b>                | $36.3 \pm 0.7$                    |
| CRL        | NN-MPC         | 56.0%                       | $40.7 \pm 0.4$                    |
| JEPA       | NN             | 43.0%                       | $43.1 \pm 0.5$                    |
| JEPA       | BC             | 15.5%                       | $28.2 \pm 0.4$                    |
| JEPA       | NN-Bias        | 15.0%                       | $30.8 \pm 0.4$                    |
| JEPA       | BC-Bias        | 15.0%                       | $29.1 \pm 0.3$                    |
| JEPA       | NN-MPC         | 44.5%                       | $65.0 \pm 2.0$                    |

Table 3: Gridworld evaluation over 200 rollouts. Successful steps reports mean  $\pm$  standard error over successful rollouts only. The method with highest success rate is bolded.

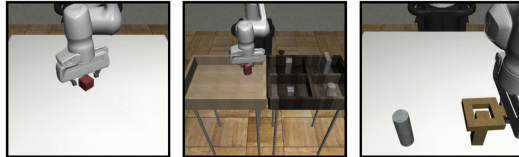


Figure 4: Robomimic tasks (Lift, Can, Square from left to right) from [12].

As for BYOL, it is expected that NN performs better than BC since this is also what was generally found in the original VINN paper [11]. The biasing methods also reduce success rate because BYOL lacks the Gaussian latent geometry that is assumed in our methods.

As for JEPA, the poor results are largely due to the unstable representation learning, which produced embeddings that collapsed into only two dimensions. Future work may explore how to make JEPA training more stable and less hyperparameter-dependent.

## E Robomimic Task Visualizations

Here we visualize the three Robomimic tasks used in the paper, as well as some intermediate results and final rollouts illustrating the effectiveness of our method on these tasks. The tasks are depicted in Figure 4 and brief descriptions are as follows:

- **Lift:** A robot arm with a 1-DoF gripper, simulated in MuJoCo, must pick up a red block randomly oriented on a table and lift it above a certain height threshold.
- **Can:** A robot arm with a 1-DoF gripper, simulated in MuJoCo, must pick up a can randomly placed on a table and move it to a receptacle on the other side of the table, then release it.
- **Square:** A robot arm with a 1-DoF gripper, simulated in MuJoCo, must pick up an irregularly-shaped object with a square hole from a table, move it to a square peg on the other side of the table, then release it.

Next we visualize various stages of our method on the simple Lift environment – the results look similar on the other two Robomimic environments. First, we visualize the high-dimensional embeddings produced by our three representation methods by projecting them down onto a 2-dimensional PCA, seen in Figure 5. Plotted points vary in color as they progress from the beginning of demon-

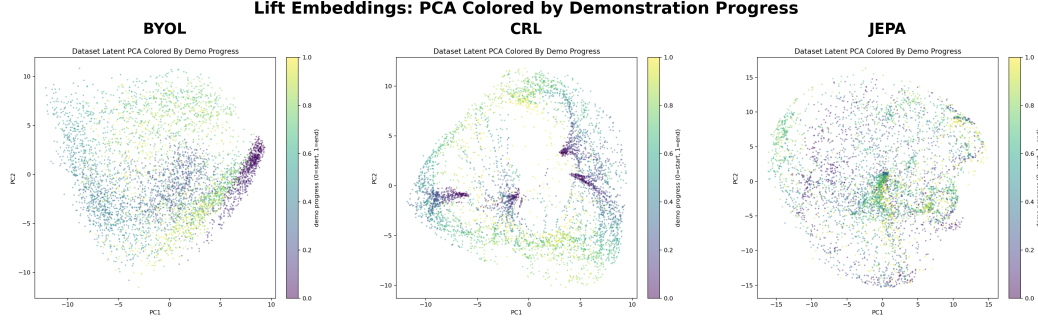


Figure 5: 2D PCA projections of the latent embeddings for a sample of Lift task demonstration datapoints, for BYOL, CRL, and JEPA representation methods. Points are colored by how far along they are in their demonstration (purple points are start states, yellow points are end states).

strations to the end, showing temporal locality strongly in CRL while less strongly in the visual representations of BYOL. We can see most clearly in CRL embeddings the expected “ring” shape, first sketched in Figure 1. Demonstrations progress along the ring from purple starts to yellow endings, with the sparse central area potentially corresponding to out-of-distribution data. The JEPA embeddings seem less clear, but this may just be because they are a projection of a 256-dimensional latent onto 2 dimensions, causing much of the nuance to be lost.

To examine whether the embeddings really maintain an isotropic Gaussian geometry, we compute various Gaussian diagnostics in Figure 6. We see that both CRL and JEPA have a diagonal covariance matrix and norms clustered around a single value, the safety sphere radius  $\hat{R}$  – exactly as our theory predicted. JEPA in particular, though its PCA appears to lack structure, has very concentrated norms, though the per-dimension means of its embeddings are farther from 0 than would be expected of an isotropic Gaussian. BYOL, on the other hand, is completely lacking in these features due to the lack of isotropic Gaussian regularization. These results also hold for the other two Robomimic environments, providing strong evidence that our representation methods have the intended geometric effects.

Now that we have qualitatively seen that the *in-distribution* embeddings lie on the safety sphere, it remains to be seen whether the *out-of-distribution* embeddings do not lie on the safety sphere. We explore this by investigating successful and failed rollouts in Figure 7. The successful rollouts move along the safety sphere in the same way as the demonstrations, evidenced by the norm remaining relatively constant. The failed rollouts typically diverge at some point, gradually sliding into the interior of the sphere or out to the exterior of the sphere, as we predicted in Figure 1. The norm shrinks far below  $\hat{R}$  or increases far above  $\hat{R}$ . This suggests that, at least to some extent, out-of-distribution states are located outside of the safety sphere, supporting our hypothesis about how the latent space is arranged. The result holds for the other Robomimic environments, and matches the aggregated data we discussed in Table 2.

If we believe the safety sphere to represent in-distribution states, and the region outside the sphere to represent out-of-distribution states, the remaining question is whether our biasing actually helps us to stay on the safety sphere. Though in aggregate we see (from Table 1) that the biasing effect is not significant, in individual rollouts for each environment we can see that  $a_{\text{bias}}$  does its job. Frames of representative rollouts are included for each environment in Figure 8 (the full rollout GIFs, and several others, are linked in Appendix A). We see that on these rollouts, original VINN (BYOL-NN) and our best unbiased method (CRL-BC) both falter, failing to pick up objects completely due to compounding distribution shift. In CRL-BC, CRL embedding norm either increases significantly above  $\hat{R}$  or decreases significantly below  $\hat{R}$  in every rollout. But when bias is added to this method, rollouts from the exact same start configuration now experience no compounding distribution shift, the  $\|z\|/\hat{R}$  remaining consistently near 1. As we had planned, the bias force increases when the state begins to move away from the safety sphere, pushing it back into the sphere, which results in

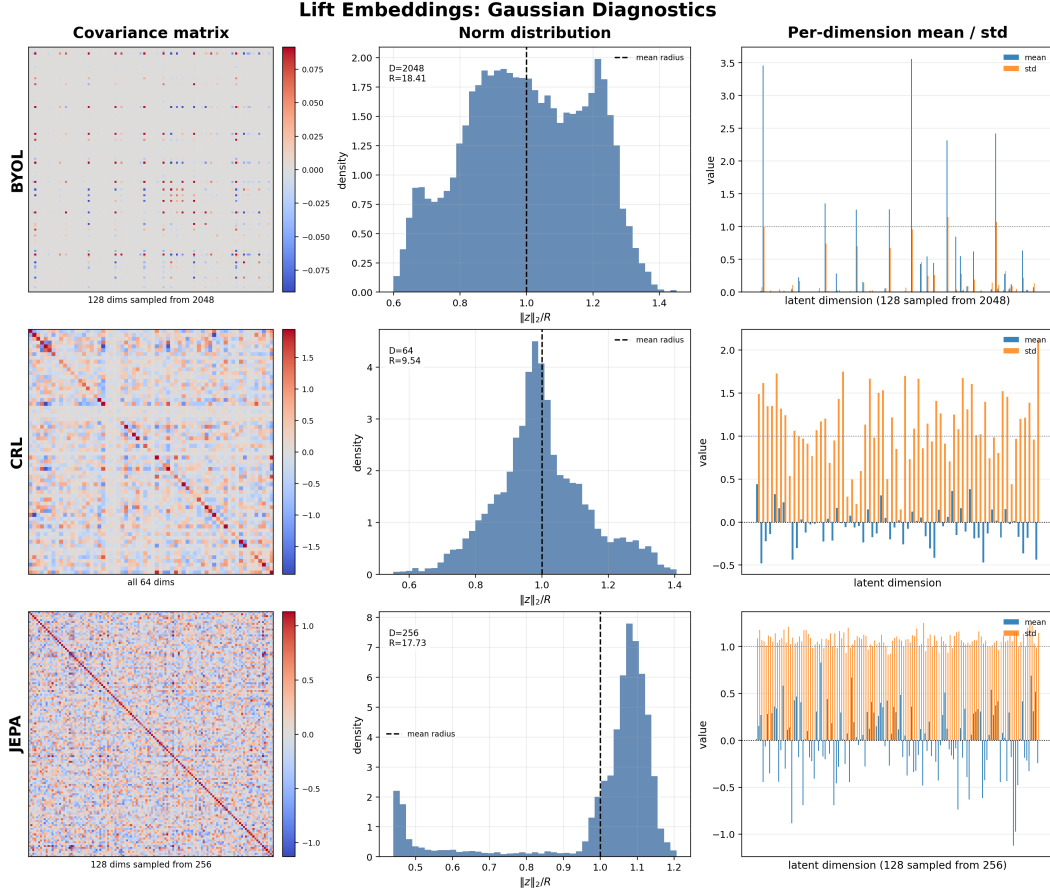


Figure 6: Tables showing Gaussian features of the latent embeddings for a sample of Lift task demonstration datapoints, for BYOL (top row), CRL (middle row), and JEPA (bottom row) representation methods. Left column depicts covariance matrix heatmaps for the embeddings (which we would expect to be near identity in an isotropic Gaussian). Middle column depicts histogram of embedding norms relative to  $\hat{R}$  (which we would expect to look like a chi-square distribution in an isotropic Gaussian). Right column depicts per-dimension means and standard deviations of embeddings, which we would expect to be 0 and 1 respectively in an isotropic Gaussian.

a flat norm curve. And in real space, this corresponds to successfully picking up and maneuvering the object. It is clear from examples like these that our biasing methods at least have some effect on extremal cases, even if they do not have a significant effect on average.

## F Exploring Policy Failures

**Why JEPA Representations Perform Worse than CRL Representations.** We explored the surprising poor performance of policies using JEPA representations, compared to those using CRL representations. There is no clear way to evaluate the quality of a set of representations, but we take inspiration from Balestriero and LeCun [9] by combining notions of linear probing, nonlinear probing (kNN  $R^2$ ), and temporal monotonicity into a single “selection score”. Linear probing and nonlinear probing come directly from Balestriero and LeCun [9], and check how useful a set of embeddings is for types of downstream tasks. Temporal monotonicity is our own score, reflecting how correlated the embeddings of nearby states in a trajectory are. We also add our own “Gaussian score” to investigate how isotropic Gaussian the embedding space is, by summing up various penalties (deviation of the average embedding from 0, deviation of the embedding variance from a

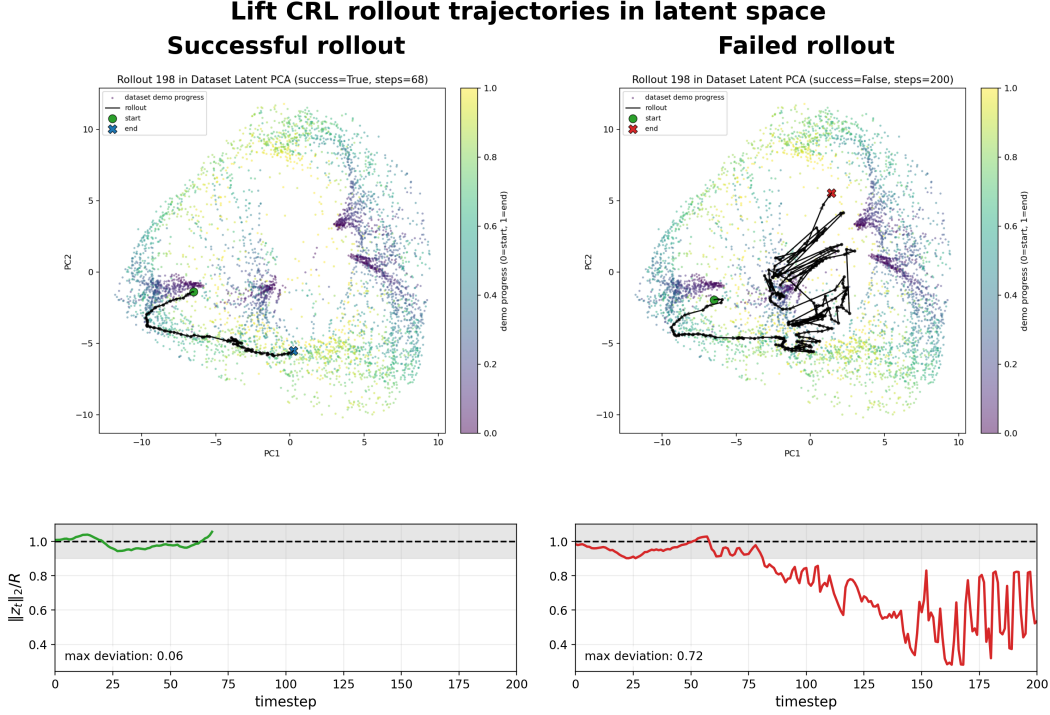


Figure 7: Plots of a successful BC policy rollout and a failed BC policy rollout in the CRL representation space of the Lift environment (matching the representation seen in Figure 5). Top graphs show the trajectory in the 2D PCA of the latent space, with black arrows connecting the start (green circle) to the end (blue cross if success, red cross if failure). Bottom graphs show the trajectory in the 1D representation of latent norm, with the calculated safety sphere radius represented as a horizontal dotted line.

| Metric                | Lift  |              |              | Can   |              |       | Square |              |       |
|-----------------------|-------|--------------|--------------|-------|--------------|-------|--------|--------------|-------|
|                       | BYOL  | CRL          | JEPA         | BYOL  | CRL          | JEPA  | BYOL   | CRL          | JEPA  |
| Selection score       | 0.481 | <b>0.595</b> | 0.386        | 0.320 | <b>0.619</b> | 0.461 | 0.430  | <b>0.664</b> | 0.534 |
| Linear probe $R^2$    | 0.381 | <b>0.495</b> | 0.340        | 0.000 | <b>0.544</b> | 0.403 | 0.189  | <b>0.572</b> | 0.545 |
| kNN $R^2$             | 0.504 | <b>0.527</b> | 0.356        | 0.516 | <b>0.585</b> | 0.440 | 0.564  | <b>0.663</b> | 0.561 |
| Temporal monotonicity | 0.666 | <b>0.938</b> | 0.542        | 0.697 | <b>0.848</b> | 0.627 | 0.736  | <b>0.870</b> | 0.463 |
| Gaussian score        | 0.000 | 0.126        | <b>0.300</b> | 0.000 | <b>0.302</b> | 0.137 | 0.000  | <b>0.363</b> | 0.228 |

Table 4: Embedding quality metrics across Robomimic tasks. The best scores within each environment are bolded.

chi-square distribution, deviation of the off-diagonal covariances from 0). Specific implementation details can be seen in the code in Appendix A.

Table 4 contains the scores for each embedding method on each task. CRL is the best in every aspect of the selection score, potentially explaining why its policies are so superior – its embeddings simply explain the space better than BYOL or JEPA can. The limitation of BYOL is primarily in linear probing and temporal monotonicity, likely reflecting the fact that BYOL is not trained on temporal information. The limitation of JEPA is in all aspects, suggesting that the embeddings may not be well trained besides meeting the Gaussian regularization; in practice, we found JEPA difficult to tune and often unstable. As for the Gaussian score, it is predictably 0 for all the BYOL embeddings, since they have no structural regularization. JEPA is more Gaussian on the simple Lift environment, but CRL is more Gaussian on the challenging environments. The especially high Gaussian score of CRL on the Square environment might explain why biasing is most effective for CRL on Square –

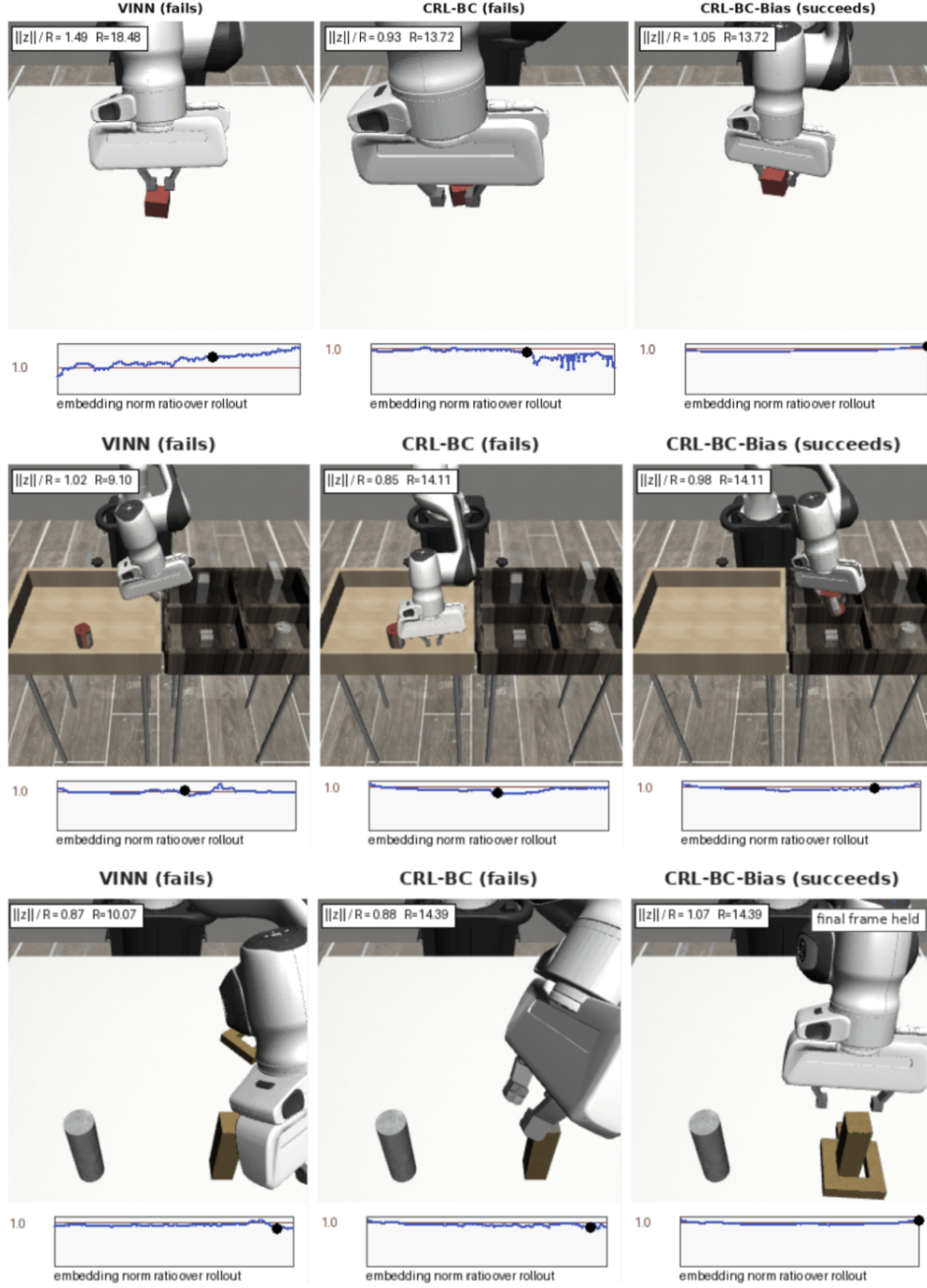


Figure 8: Frames from rollouts on Lift (top), Can (middle), and Square (bottom) tasks. The left column consists of VINN (BYOL-NN) rollouts, the middle column consists of CRL-BC rollouts, and the right column consists of CRL-BC-Bias rollouts. Embedding norm ratio  $\|z\|/\hat{R}$  is plotted below each frame, with the current norm labeled with a dot and printed in the top left. Full videos of these rollouts can be found via the link in Appendix A.

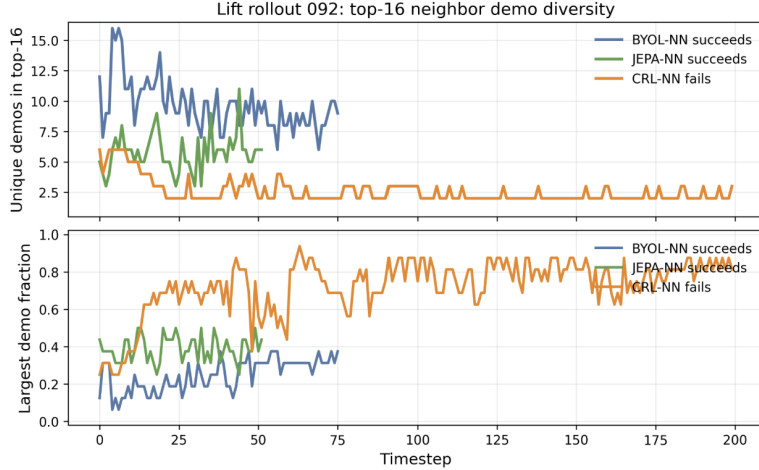


Figure 9: Comparison of demonstration diversity among the top- $k$  neighbors at every timestep of a representative rollout for the three embedding methods. The top plot shows the number of unique demos represented by the  $k$  points (the lower this is, the more concentrated the information), the bottom plot shows the largest fraction of the  $k$  points that belongs to one demonstration (the higher this is, the more concentrated the information).

the more Gaussian the embedding, the more concentrated the norms around the safety sphere, and the better it is to bias toward that sphere.

**Why NN Performs Worse on CRL Representations.** We explored the surprising poor performance of NN relative to BC on CRL embeddings, compared to the BYOL and JEPA embeddings which tend to perform well with NN. From visual inspection of CRL-NN rollouts, we saw a tendency to “miss” grasping the object entirely, often closing the gripper before reaching the object and then finishing the demonstration as if the object was held. This was much less prevalent when demonstrations were imitated with BC.

We hypothesized that this could be due to the CRL latent space containing much larger distances *between* demonstrations than *within* demonstrations. If this was true, the CRL-NN policy would only take neighbors from a few demonstrations, increasing the chance that it follows a misleading one.

Figure 9 provides evidence for this hypothesis: it plots the number of unique demonstrations represented by the  $k = 16$  neighbors used in a CRL-NN rollout, and finds that at around the timestep when CRL diverges from JEPA and BYOL, this number is as low as 2, while the other two methods use 5 or 10 demonstrations. Indeed, due to the distribution of latent space distances, CRL-NN only observes actions from a small number of trajectories, making it much more likely to get stuck. Figure 10 shows this statistic averaged over all 200 rollouts, with the general trends supporting the same conclusion. This suggests that there is an additional, more subtle necessity for embeddings to work with NN action selection – their demonstrations must not be separated too greatly in latent space. This is easy for BYOL to achieve, since its positive pairs are only semantic and do not use temporal information. But this is challenging for CRL to achieve, since its positive pairs all come from the same trajectory, so it is encouraged to cluster trajectories together. Hence, CRL – and to a lesser extent JEPA – will generally struggle with the action selection method that worked so well in VINN.

**Why NN-MPC Performs Worse than NN.** We explored the surprising poor performance of NN-MPC relative to NN on CRL/JEPA embeddings. It is especially noticeable on JEPA for Lift, where the success rate with NN is around 30% while the success rate with NN-MPC is only around 10%; despite the NN-MPC method being built to improve reliability, it actually decreases it. From visual inspection of JEPA-NN-MPC rollouts, we saw a tendency to stall before grasping the object,

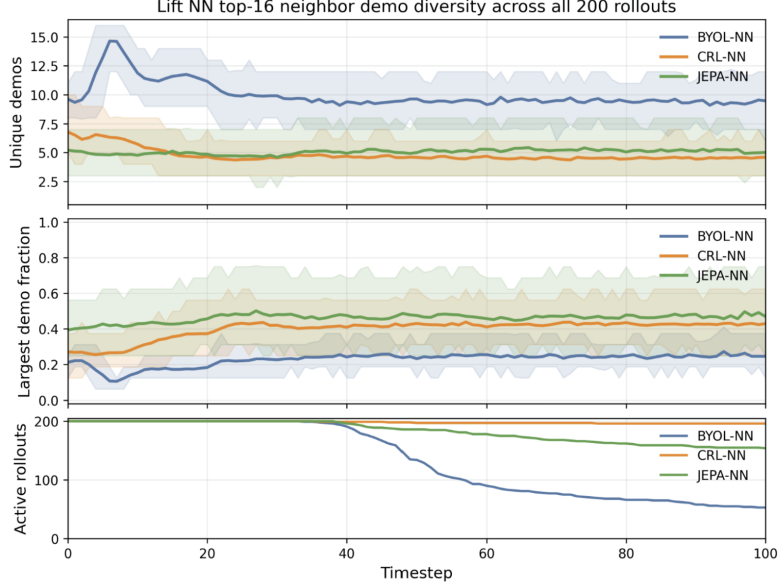


Figure 10: Top two plots are equivalent of Figure 9, averaged over all 200 eval rollouts. Bottom plot shows the number of active rollouts at this timestep – the lower this number, the more rollouts have already succeeded (BYOL is successful, CRL is not).

hovering around the object for a long time and often outlasting the step limit. This was much less prevalent when the same rollouts were attempted with JEPA-NN.

We hypothesized that this could be due to the action weightings in NN-MPC being too uniform. If the weightings are uniform, and the actions disagree, they would add to nearly 0, causing small and ineffective output actions. The weightings in NN-MPC only change dramatically when some of the neighbors are perceived as “risky”, but if no neighbors seem risky then there is no information about which neighbor to follow.

Figure 11 provides evidence for this hypothesis. The JEPA-NN rollout quickly converges because it generally has a neighbor action with over half of the weight; on the other hand, the JEPA-NN-MPC rollout hovers for the whole rollout because the maximum action weight is often under 0.3, and the minimum action weight is notably greater than 0. Figure 12 shows this statistic averaged over all 200 rollouts, with the general trends supporting the same conclusion. This suggests that the local distance weightings in VINN are not superfluous, but are actually critical in more challenging environments, allowing the agent to strongly upweight close neighbor actions. By removing this distance weighting, NN-MPC produces nearly uniform weights in most cases, leading to hesitant and ineffective policies. The results also suggest that NN-MPC could be improved by finding a way to incorporate from both distance and riskiness – even summing or multiplying the scores could potentially produce less uniform weights than NN-MPC alone, though we do not test this ablation in our work.

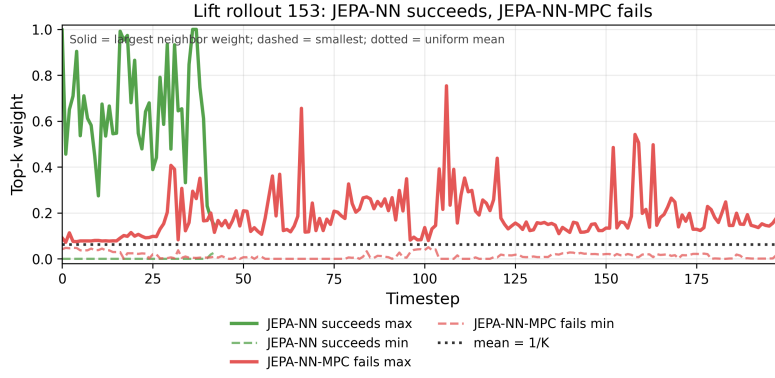


Figure 11: Comparison of maximum (solid) and minimum (dashed) SoftMin weights assigned to the top- $k$  neighbor actions at every timestep of a Lift rollout with JEPA embeddings. Green rollout uses NN action selection and succeeds, red rollout uses NN-MPC action selection and fails. Horizontal dashed line represents the mean weight of  $1/k$ .

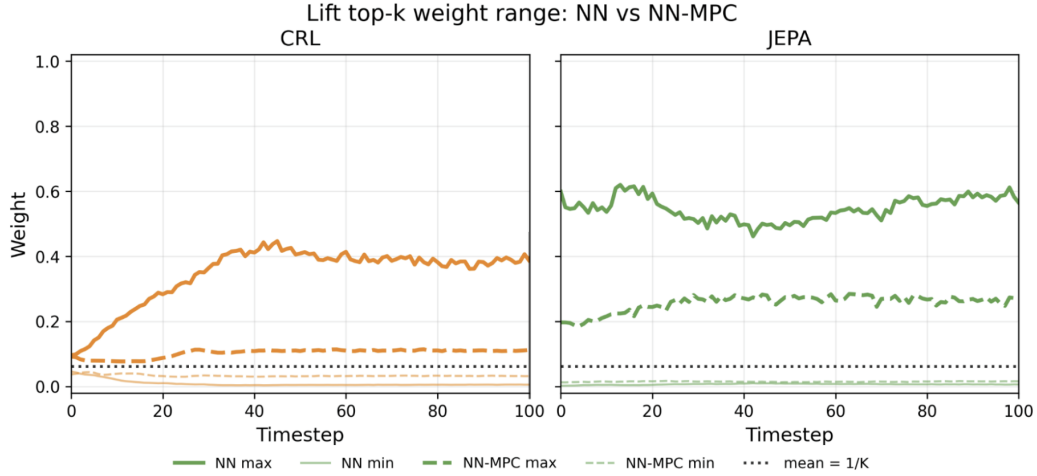


Figure 12: Right plot is the equivalent of Figure 11, averaged over all 200 eval rollouts on JEPA representations. Left plot is the equivalent of this, for CRL representations. Thick lines are maximum top- $k$  weights, thin lines are minimum top- $k$  weights; solid are NN weights, dashed are NN-MPC weights.